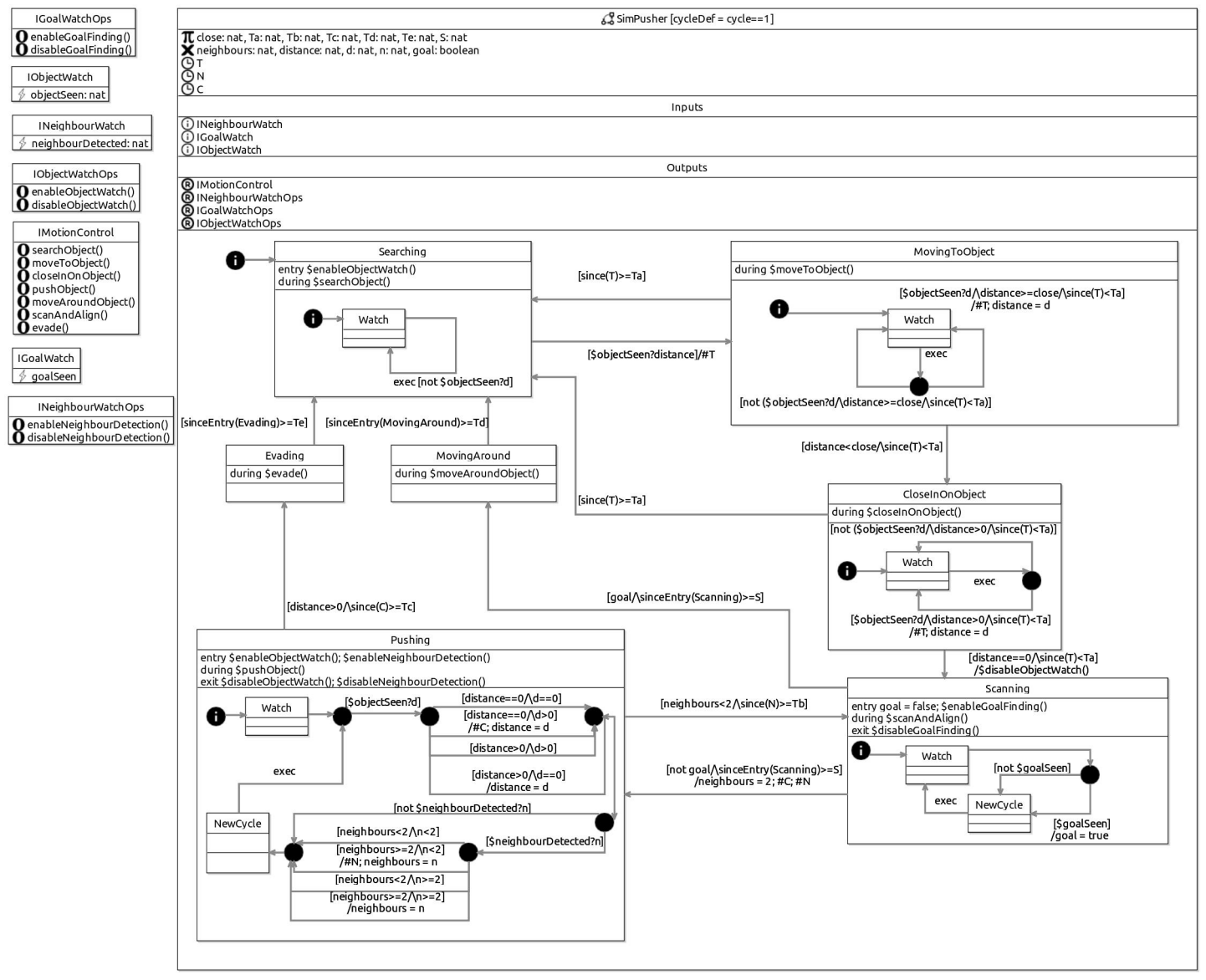

IGoalWatchOps
enableGoalFinding()
disableGoalFinding()
IObjectWatch
objectSeen: nat
INeighbourWatch
neighbourDetected: nat
IObjectWatchOps
enableObjectWatch()
disableObjectWatch()
IMotionControl
searchObject()
moveToObject()
closeInOnObject()
pushObject()
moveAroundObject()
scanAndAlign()
evade()
IGoalWatch
goalSeen
INeighbourWatchOps
enableNeighbourDetection()
disableNeighbourDetection()
SimPusher [cycleDef = cycle==1]
close: nat, Ta: nat, Tb: nat, Tc: nat, Td: nat, Te: nat, S: nat
neighbours: nat, distance: nat, d: nat, n: nat, goal: boolean
T
N
C
Inputs
INeighbourWatch
IGoalWatch
IObjectWatch
Outputs
IMotionControl
INeighbourWatchOps
IGoalWatchOps
IObjectWatchOps
Searching
entry $enableObjectWatch()
during $searchObject()
Watch
exec [not $objectSeen?d]
[since(T)>=Ta]
[$objectSeen?distance]/#T
MovingToObject
during $moveToObject()
[$objectSeen?d/\distance>=close/\since(T)<Ta]
/#T; distance = d
Watch
exec
[not ($objectSeen?d/\distance>=close/\since(T)<Ta)]
[sinceEntry(Evading)>=Te]
[sinceEntry(MovingAround)>=Td]
Evading
during $evade()
MovingAround
during $moveAroundObject()
[distance<close/\since(T)<Ta]
[since(T)>=Ta]
CloseInOnObject
during $closeInOnObject()
[not ($objectSeen?d/\distance>0/\since(T)<Ta)]
Watch
exec
[$objectSeen?d/\distance>0/\since(T)<Ta]
/#T; distance = d
[distance>0/\since(C)>=Tc]
[goal/\sinceEntry(Scanning)>=S]
[distance==0/\since(T)<Ta]
/$disableObjectWatch()
Pushing
entry $enableObjectWatch(); $enableNeighbourDetection()
during $pushObject()
exit $disableObjectWatch(); $disableNeighbourDetection()
Watch
[$objectSeen?d]
[distance==0/\d==0]
[distance==0/\d>0]
/#C; distance = d
[distance>0/\d>0]
[distance>0/\d==0]
/distance = d
exec
[neighbours<2/\since(N)>=Tb]
Scanning
entry goal = false; $enableGoalFinding()
during $scanAndAlign()
exit $disableGoalFinding()
Watch
[not $goalSeen]
exec
NewCycle
[$goalSeen]
/goal = true
[not goal/\sinceEntry(Scanning)>=S]
/neighbours = 2; #C; #N
[not $neighbourDetected?n]
NewCycle
[neighbours<2/\n<2]
[neighbours>=2/\n<2]
/#N; neighbours = n
[$neighbourDetected?n]
[neighbours<2/\n>=2]
[neighbours>=2/\n>=2]
/neighbours = n